

cartographer Mapping Algorithm

cartographer Mapping Algorithm

1. Course Content
2. Cartographer Introduction
3. Running the Demo
 - 3.1 Mapping Process
 - 3.2 Saving the Map
 - 3.3 Saving a Pose Graph Map in pstream Format

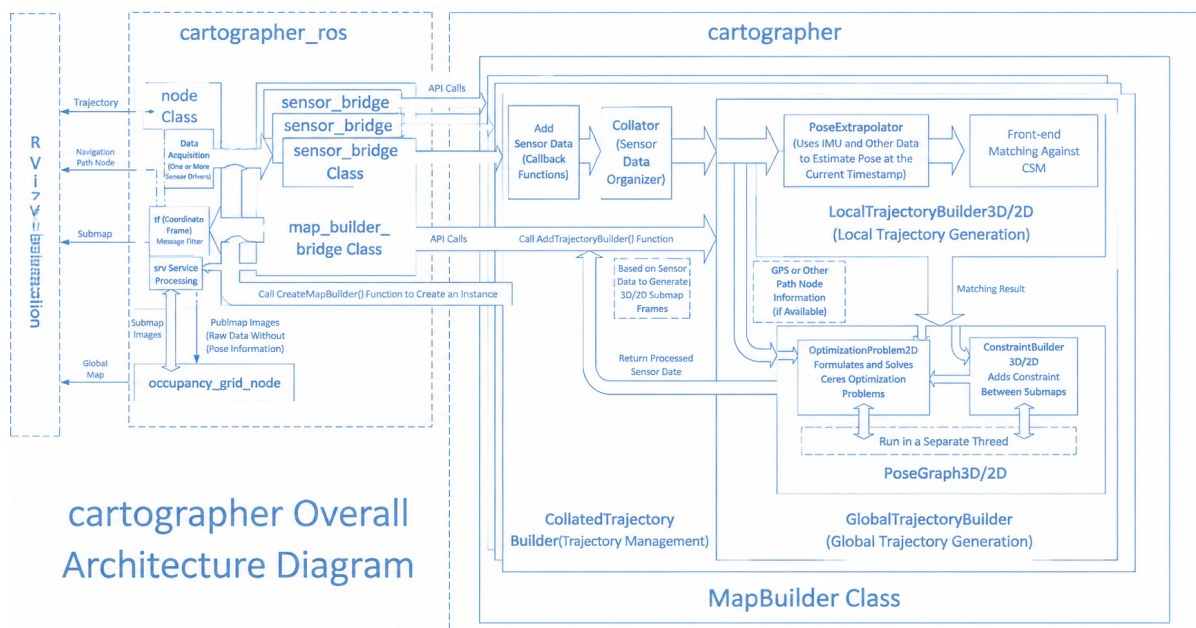
1. Course Content

1. Run the demo cartographer mapping algorithm to perform SLAM mapping, and save both the occupancy grid map and the pose graph map.

2. Cartographer Introduction

Cartographer is an open-source 2D and 3D SLAM library supporting the ROS system, released by Google. It is a mapping algorithm based on graph optimization (multi-threaded back-end optimization, problem optimization built with Ceres). It can combine data from multiple sensors (such as LiDAR, IMU, and camera) to simultaneously compute the sensor positions and map the environment around the sensors.

The source code of cartographer mainly consists of three parts: cartographer, cartographer_ros, and ceres-solver (back-end optimization).



cartographer adopts the mainstream SLAM framework, namely the three-stage structure of feature extraction, loop-closure detection, and back-end optimization. A certain number of LaserScans form a submap, and a series of submaps make up the global map. The short-term process of building a submap from LaserScans accumulates little error, but the long-term process

of building the global map from submaps accumulates significant error, so loop-closure detection is needed to correct the positions of these submaps. The basic unit of loop-closure detection is the submap, and loop-closure detection uses the scan_match strategy. The key aspects of cartographer are the creation of submaps that fuse multi-sensor data (odometry, IMU, LaserScan, etc.) and the implementation of the scan_match strategy used for loop-closure detection.

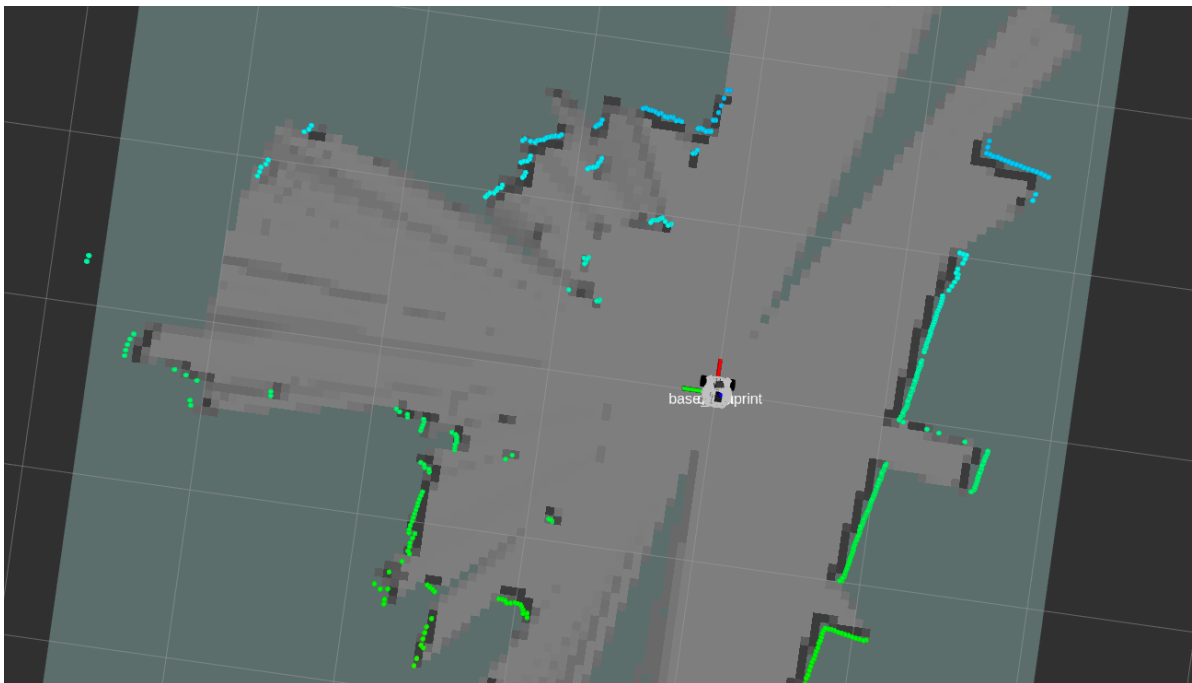
- cartographer_ros

cartographer_ros runs under ROS and can receive various sensor data in the form of ROS messages; after

processing, it publishes them again as messages for easy debugging and visualization.

3. Running the Demo

```
ros2 launch microros_v2_bringup cartographer.launch.py
```



3.1 Mapping Process

- Note: when mapping, slower speeds give better results (especially the rotation speed should be slow). If you move too fast, the mapping quality will be poor.
- Control the robot to move and complete mapping of the environment. You can use keyboard control or gamepad control; keyboard control is used here as an example.

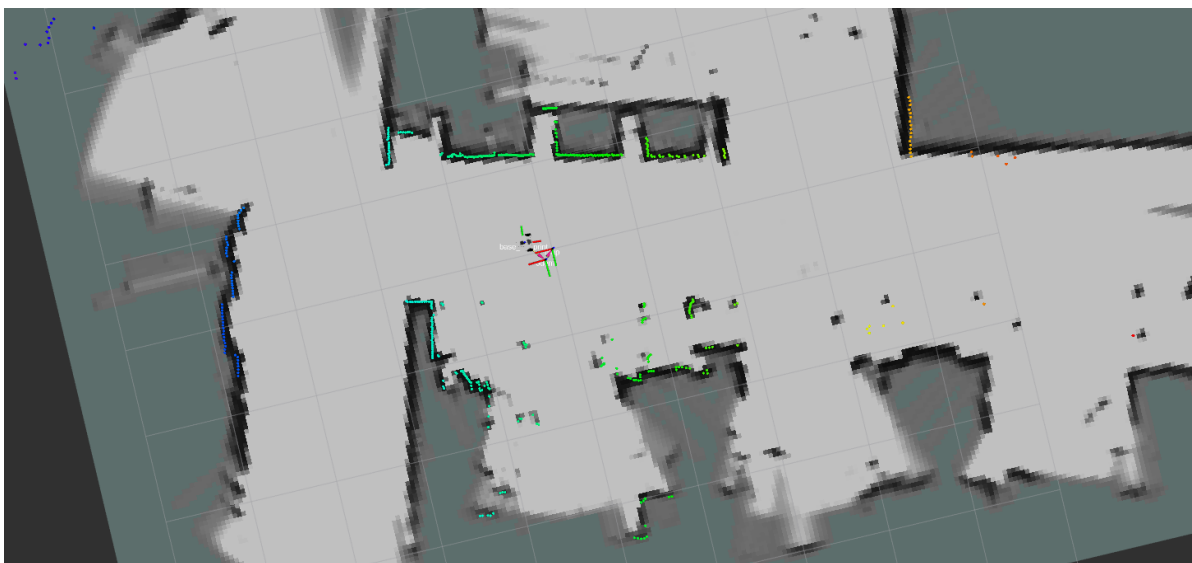
```
ros2 run common_demo keyboard_control
```

```
yahboom@yahboom:~$ ros2 run common_demo keyboard_control

Reading from the keyboard and Publishing to Twist!
-----

Moving around:
    u    i    o
    j    k    l
    m    ,    .

For Holonomic mode (strafing), hold down the shift key.
```



3.2 Saving the Map

- Run the following command in the terminal to save the occupancy grid map:

```
ros2 launch microros_v2_bringup save_map.launch.py
```

[!TIP]

Optional parameters:

`map_name`: specify the name of the map to save; the default value is map

For example, to save a map named `test`, add the parameter: `ros2 launch microros_v2_bringup save_map.launch.py map_name:=test`

A **Map saved successfully** message in the terminal means the map was saved successfully. If saving fails, try again.

- The map save path is as follows:

```
~/map/map.yaml
```

- One PGM image and one YAML file, map.yaml

```
image: map.pgm
mode: trinary
resolution: 0.05
origin: [-10, -10, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

Parameter explanations:

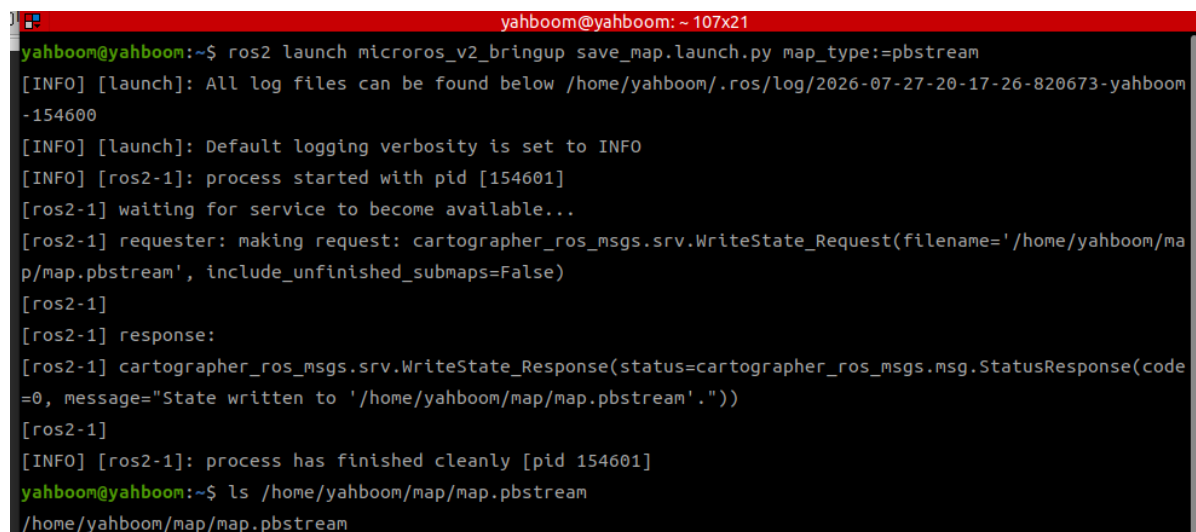
- image: path to the map file; it can be an absolute path or a relative path
- mode: this attribute can be one of trinary, scale, or raw, depending on the mode chosen; trinary is the default mode
- resolution: the resolution of the map, in meters/pixel
- origin: the 2D pose (x, y, yaw) of the bottom-left corner of the map. Here yaw is rotated counterclockwise (yaw=0 means no rotation). Many parts of the current system ignore the yaw value.
- negate: whether to invert the meaning of white/black and free/occupied (the interpretation of the thresholds is not affected)
- occupied_thresh: pixels with an occupancy probability greater than this threshold are considered fully occupied.
- free_thresh: pixels with an occupancy probability less than this threshold are considered completely free.

3.3 Saving a Pose Graph Map in pbstream Format

Map files in pbstream format are used for cartographer localization and navigation, which will be explained in a later chapter.

```
ros2 launch microros_v2_bringup save_map.launch.py map_type:=pbstream
```

- Pose graph files are saved under `~/map/` by default



```
yahboom@yahboom: ~ 107x21
yahboom@yahboom:~$ ros2 launch microros_v2_bringup save_map.launch.py map_type:=pbstream
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-27-20-17-26-820673-yahboom-154600
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [ros2-1]: process started with pid [154601]
[ros2-1] waiting for service to become available...
[ros2-1] requester: making request: cartographer_ros_msgs.srv.WriteState_Request(filename='/home/yahboom/map/map.pbstream', include_unfinished_submaps=False)
[ros2-1]
[ros2-1] response:
[ros2-1] cartographer_ros_msgs.srv.WriteState_Response(status=cartographer_ros_msgs.msg.StatusResponse(code=0, message="State written to '/home/yahboom/map/map.pbstream'."))
[ros2-1]
[INFO] [ros2-1]: process has finished cleanly [pid 154601]
yahboom@yahboom:~$ ls /home/yahboom/map/map.pbstream
/home/yahboom/map/map.pbstream
```

主文件夹 / map



map.
pbstream



map.pgm



map.yaml